Magna

Security Assessment

March 19th, 2025 — Prepared by OtterSec

Bartłomiej Wierzbiński dark@osec.io

Robert Chen notdeghost@osec.io

Table of Contents

Executive Summary **3**

Overview 3

Key Findings 3

Scope **4**

Findings **5**

Vulnerabilities **6**

OS-MGN-ADV-00 | Frontrunning Vesting Creation 7

OS-MGN-ADV-01 | Incorrect Rejections of Cancellation Requests 9

OS-MGN-ADV-02 | Failure to Check Schedules Funding Status 11

OS-MGN-ADV-03 | Improper Termination Handling in Withdrawals 12

General Findings **13**

OS-MGN-SUG-00 | Unverifiable Merkle Proof due to Length Mismatch 15

OS-MGN-SUG-01 | Missing Validation Logic 17

OS-MGN-SUG-02 | Additional Safety Checks 19

OS-MGN-SUG-03 | Error Handling 21

OS-MGN-SUG-04 | Utilization of incorrect Comparison operators 22

OS-MGN-SUG-05 | Code Maturity 24

OS-MGN-SUG-06 | Code Redundancy 26

OS-MGN-SUG-07 | Unutilized Code 28

OS-MGN-SUG-08 | Unnecessary Assert Check 30

OS-MGN-SUG-09 | Potential Overflow Scenarios 32

© 2025 Otter Audits LLC. All Rights Reserved.

TABLE OF CONTENTS

Appendices

Vulnerability Rating Scale 33

Procedure 34

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Magna engaged OtterSec to assess the `airlock-aptos` program. This assessment was conducted between February 28th and March 17th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 14 findings throughout this audit engagement.

We identified several vulnerabilities in the vesting creation and management process. The vesting creation mechanism is susceptible to frontrunning attacks, allowing attackers to preemptively create a vesting contract with the same seed ID or register an account for the target vesting address, resulting in legitimate creation attempts to fail (OS-MGN-ADV-00). Additionally, the cancellation of interval-based vesting schedules incorrectly determines full unlock status by disregarding the cliff timestamp. If the cliff timestamp is set after the last period, the function may abort, preventing valid cancellations (OS-MGN-ADV-01). Furthermore, `can_fund` does not check if the schedule is already fully funded and may falsely claim that funding is possible when it is not (OS-MGN-ADV-02).

We also made suggestions regarding the need to ensure adherence to coding best practices and code optimization (OS-MGN-SUG-05), and suggested removing redundant and unutilized code instances (OS-MGN-SUG-06, OS-MGN-SUG-07). Moreover, we advised including additional safety checks within the codebase to improve security (OS-MGN-SUG-01,OS-MGN-SUG-03). Lastly, when generating a Merkle proof, the program inserts an empty vector instead of duplicating the last node for odd-length levels, resulting in unverifiable proofs due to length mismatches (OS-MGN-SUG-00).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/magna-eng/protocol/tree/main/move. This audit was performed against commit 679af5e0.

A brief description of the programs is as follows:

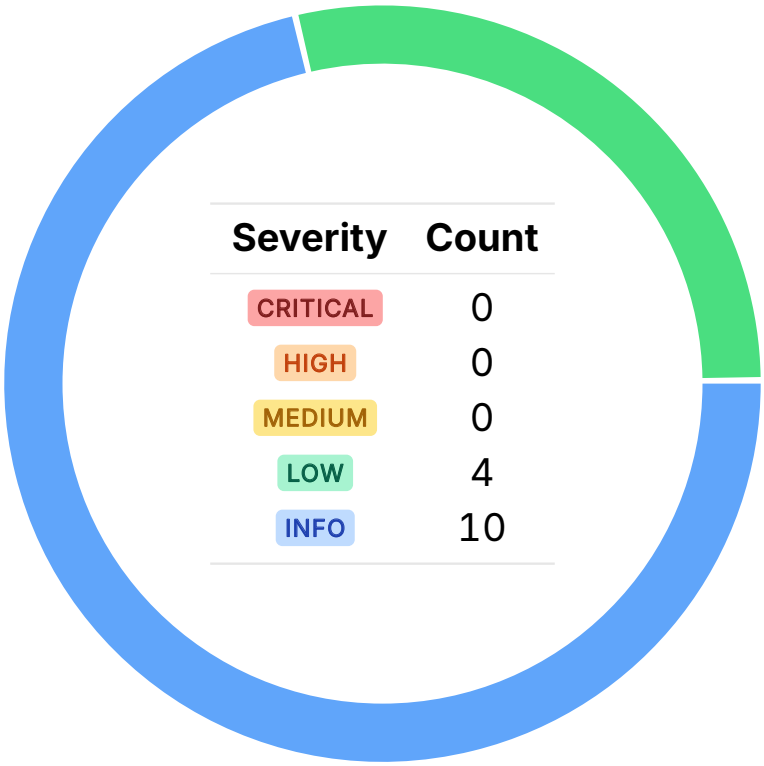
Name	Description
airlock-aptos	A rewrite of the Airlock V2 Solidity contracts in Aptos Move, including the porting of all unit tests to verify the correctness of the new implementation.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 14 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	4
INFO	10

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MGN-ADV-00	LOW	RESOLVED	`create_vesting` in `vesting` and `vesting_merkle` is vulnerable to frontrunning attacks. Attackers may preemptively create a vesting contract with the same `seed_id` or preemptively call `create_account` for the target vesting address, resulting in legitimate attempts to create a vesting contract to fail.
OS-MGN-ADV-01	LOW	RESOLVED	`cancel_schedule_interval` could incorrectly determine full unlock status by ignoring `cliff_timestamp`. If `cliff_timestamp` is set after the last period, the function may prematurely abort, preventing valid cancellations.
OS-MGN-ADV-02	LOW	RESOLVED	`can_fund` does not check if the schedule is already fully funded and may falsely claim that funding is possible when it is not.
OS-MGN-ADV-03	LOW	RESOLVED	`withdraw_interval` and `withdraw_calendar` do not explicitly handle withdrawals after revocation, resulting in an error. In addition, `can_withdraw_calendar` and `can_withdraw_interval` throw an error on revocation instead of returning true or false.

© 2025 Otter Audits LLC. All Rights Reserved.

Frontrunning Vesting Creation LOW OS-MGN-ADV-00

Description

`vesting` and `vesting_merkle` are susceptible to denial-of-service attacks by allowing an attacker to frontrun the creation of a vesting contract in multiple ways. The first vulnerability occurs due to the use of the `seed_id` in both `vesting::create_vesting` and `vesting_merkle::create_vesting_internal`. The logic in these functions contains an assertion that checks if a vesting contract already exists for a given `seed_id`, throwing an error (`ERR_VESTING_EXISTS`) if it does, preventing the contract creation from continuing.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
fun create_vesting<CoinType, PaymentCoinType>(seed_id: vector<u8>,[...]): Object<Vesting>
    ↪  acquires Airlock {
    [...]
    assert!(!vesting_exists(seed_id), ERR_VESTING_EXISTS);
    [...]
    aptos_account::create_account(vesting_addr);
}
```

However, since anyone may create a vesting contract with any `seed_id`, enabling an attacker to frontrun a `create_vesting` call with their own call to `create_vesting` utilizing the same `seed_id`. This will abort the original user's vesting creation.

>_ *move/airlock_vesting/sources/vesting_merkle.move* RUST

```rust
public fun create_vesting_internal<CoinType, PaymentCoinType>(seed_id: vector<u8>,[...]):
    ↪  Object<Vesting> acquires Airlock {
    [...]
    assert!(!vesting_exists(seed_id), ERR_VESTING_EXISTS);
    [...]
    aptos_account::create_account(vesting_addr);
}
```

Similarly, both `vesting::create_vesting` and `vesting_merkle::create_vesting_internal` create an Aptos account via `create_account`. `create_account` may be called by anyone with any address and will abort if the account already exists. Consequently, an attacker may frontrun the vesting creation by preemptively calling `create_account` for the intended address, resulting in that users `create_vesting` call to fail.

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Utilize object creation functions which do not rely on predefined seeds. Additionally, `create_vesting` should skip account creation if the account already exists.

Patch

This issue was acknowledged by the client and the risk was accepted.

© 2025 Otter Audits LLC. All Rights Reserved.

Incorrect Rejections of Cancellation Requests LOW OS-MGN-ADV-01

Description

`vesting::cancel_schedule_interval` determines whether an interval-based vesting schedule may be canceled. One of its key checks ensures that the schedule is not already fully unlocked before allowing cancellation. However, the function currently does not consider the `cliff_timestamp` while checking the fully unlocked status. This omission may result in a logic error where valid schedules may be incorrectly considered fully unlocked and thus non-cancelable.

`fully_unlocked_timestamp` is calculated assuming that full unlocking occurs immediately after the last period ends, without considering the `cliff_timestamp`, which defines the earliest time at which any amount beyond `unlocked_at_start` may be accessed.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
public entry fun cancel_schedule_interval([...]) acquires Interval, Schedule, Vesting {
    [...]
    let fully_unlocked_timestamp =
        interval.unlock_timestamp
            + (interval.number_of_periods * interval.period_length);
    let now_seconds = timestamp::now_seconds();
    assert!(now_seconds <= fully_unlocked_timestamp, ERR_ALREADY_FULLY_UNLOCKED);
    [...]
    event::emit(ScheduleCanceledEvent { schedule_addr });
}
```

Thus, in a case `cliff_timestamp` is set to timestamp after the last period end, and `now_seconds` is between the last vesting period end and `cliff_timestamp`, `cancel_schedule_interval` will abort with `ERR_ALREADY_FULLY_UNLOCKED` as `now_seconds` is greater than `fully_unlocked_timestamp`, despite the fact that only `unlocked_at_start` amount is unlocked at this point. As a result, the cancellation of the vesting schedule will be restricted even though the majority of the funds are still locked due to the `cliff_timestamp`.

Remediation

Ensure that the vesting contract design enforces `cliff_timestamp` not exceeding `fully_unlocked_timestamp`. If this condition is not met, the schedule creation should be rejected to prevent `cliff_timestamp` from extending beyond the final period end, thereby eliminating this issue. However, if the case of `cliff_timestamp` after last period ends is valid, it should be taken into consideration while checking the fully unlocked status in `cancel_schedule_interval`.

© 2025 Otter Audits LLC. All Rights Reserved.

Patch

Resolved in ac03e9e.

© 2025 Otter Audits LLC. All Rights Reserved.

Failure to Check Schedules Funding Status LOW OS-MGN-ADV-02

Description

There is a potential vulnerability in `vesting::can_fund`, where the function may incorrectly return true even if the vesting schedule is already fully funded and no more funding is allowed, as the function does not check if the schedule is already fully funded.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
#[view]
public fun can_fund(schedule_obj: Object<Schedule>, amount: u64): bool acquires Schedule {
    let schedule_addr = object::object_address(&schedule_obj);
    if (!exists<Schedule>(schedule_addr)) {
        return false
    };
    let schedule = borrow_global<Schedule>(schedule_addr);
    schedule.terminated_timestamp == 0 && amount != 0
}
```

Remediation

Add a check in `can_fund` to ensure that the schedule has not already reached its funding cap. If it has, any additional funding should be prevented.

Patch

Resolved in e300c3b.

© 2025 Otter Audits LLC. All Rights Reserved.

Improper Termination Handling in Withdrawals LOW OS-MGN-ADV-03

Description

The current implementation of `withdraw_calendar` and `withdraw_interval` in both `vesting` and `vesting_merkle` do not explicitly handle the case where a schedule has been revoked. This creates a vulnerability where the withdrawn amount may exceed the vested amount under certain conditions. `distribution_state.withdrawn` and `schedule.withdrawn` may be greater than the vested amount in case of termination. This will lead to mathematical underflow errors.

This issue also exists in `can_withdraw_calendar` and `can_withdraw_interval`, which should return true or false based on whether any vested amount remains at the time of termination, instead of throwing an error and disrupting transaction execution.

Remediation

Ensure the case of withdrawal after revoke is be explicitly checked and handled separately in `withdraw_interval` and `withdraw_calendar`. Also, modify `can_withdraw_calendar` and `can_withdraw_interval` to return true or false, even if the schedule is terminated to avoid throwing errors.

Patch

Resolved in e300c3b.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MGN-SUG-00	`get_proof` incorrectly inserts an empty vector resulting in unverifiable proofs due to length mismatches in `lt`.
OS-MGN-SUG-01	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-MGN-SUG-02	Additional safety checks may be incorporated within the codebase to make it more robust and secure.
OS-MGN-SUG-03	The error handling may be improved to avoid unexpected failures during code execution.
OS-MGN-SUG-04	The utilization of `<=` instead of `<` in the timestamp checks enable the cancellation of vesting schedules even after the schedule is already fully unlocked and there is nothing to cancel.
OS-MGN-SUG-05	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-MGN-SUG-06	The codebase contains multiple cases of redundancy that should be removed for better maintainability and clarity.
OS-MGN-SUG-07	The codebase contains multiple cases of unnecessary or unutilized code that should be removed.

© 2025 Otter Audits LLC. All Rights Reserved.

OS-MGN-SUG-08	The `ERR_INVALID_WITHDRAWAL` asserts in `withdraw_calendar`, `withdraw_interval`, and `withdraw_terminated_funds` will never fail in the current implementation.
OS-MGN-SUG-09	There are multiple areas within the codebase where an overflow may occur if the administrator were to set extremely large values.

© 2025 Otter Audits LLC. All Rights Reserved.

Unverifiable Merkle Proof due to Length Mismatch OS-MGN-SUG-00

Description

`get_proof` constructs a Merkle proof by collecting sibling nodes at each level of the Merkle tree. However, if the node is the last element in an odd-length list, it incorrectly inserts an empty vector (`vector::empty()`), resulting in an unverifiable proof. When `process_proof` calls `lt` to compare byte vector lengths (`len == vector::length(b)`), the assertion fails because the empty vector does not match the expected hash length (`vector<u8>` of fixed length). Consequently, `lt` rejects the proof due to a length mismatch.

>_ *move/airlock_vesting/sources/utils/merkle_proof.move* RUST

```rust
public fun get_proof(data: vector<vector<u8>>, node: u64): vector<vector<u8>> {
    [...]
    while (vector::length(&data) > 1) {
        [...]
        else if (node + 1 == vector::length(&data)) {
            item = vector::empty(); // Equivalent to bytes32(0) in Solidity
        }
        [...]
    };
    result
}
```

The intended behavior was likely to insert an all-zero vector, as suggested by the comment in `get_proof` . However, `hash_level`, utilized in `create_root`, duplicates a node if the number of nodes is odd, implying the *alone* node is hashed with itself during root calculation. To ensure a correct proof step in such cases, the proof element must match the hashed element. Therefore, the proof should include the node itself rather than an all-zero vector.

>_ *move/airlock_vesting/sources/utils/merkle_proof.move* RUST

```rust
fun hash_level(data: &vector<vector<u8>>): vector<vector<u8>> {
   [...]
    while (i < len) {
        let left = *vector::borrow(data, i);
        let right =
            if (i + 1 < len) { *vector::borrow(data, i + 1) }
            else { left // If there's an odd number of nodes, duplicate the last one. };
        [...]
    };
    result
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Modify `get_proof` to insert a correct element while constructing the merkle proof.

Ensure that `get_proof` is annotated with `test_only` as this function is only utilized in tests as mentioned by Magna.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-MGN-SUG-01

Description

1. Add a verification in `vesting` to check if the `Vesting` is of the correct type before borrowing `Calendar` or `Interval` in `withdraw_calendar` and `withdraw_interval` respectively, to improve error handling. This ensures that the contract does not attempt to borrow data for the wrong type of vesting schedule, thereby preventing unexpected errors during execution.

   >_ *move/airlock_vesting/sources/vesting.move* RUST

   ```rust
   public entry fun withdraw_interval<CoinType, PaymentCoinType>(
       fee_payer: &signer, schedule_obj: Object<Schedule>
   ) acquires Schedule, Interval, Vesting {
       [...]
       let interval = borrow_global<Interval>(schedule_addr);
       [...]
   }

   public entry fun withdraw_calendar<CoinType, PaymentCoinType>(
       fee_payer: &signer, schedule_obj: Object<Schedule>
   ) acquires Schedule, Calendar, Vesting {
       [...]
       let calendar = borrow_global<Calendar>(schedule_addr);
       [...]
   }
   ```

2. In the current implementation of `vesting` and `vesting_merkle`, several functions should validate if the borrowed object exists before borrowing, to avoid runtime errors. For example, `vesting_merkle::terminate_schedule` should check if `Vesting` exists before borrowing. In `vesting`, `create_schedule_calendar` and `create_schedule_interval` should check for object existence as `cancel_schedule_calendar` does.
3. `vesting::create_schedule_calendar` currently checks that `unlock_amounts` is not an empty vector and that the sum of the amounts is not 0. However, it does not check if individual values are zero. A schedule where some `unlock_amounts` are zero is meaningless because unlocking zero tokens at any point has no effect. While iterating over `unlock_amounts`, assert that each value is greater than zero.
4. Ensure `number_of_periods != 0` in `vesting::create_schedule_interval` to preventing invalid interval schedules. If `number_of_periods` is zero, it will create issues when computing the total amount to be vested over the intervals.

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Add the missing validations mentioned above.

© 2025 Otter Audits LLC. All Rights Reserved.

Additional Safety Checks

OS-MGN-SUG-02

Description

1. Modify the last condition in `vesting::can_withdraw_terminated_funds` to explicitly check whether withdrawing `terminated_amount` keeps the total below `funded_amount`.

   >_ *move/airlock_vesting/sources/vesting.move* RUST

   ```rust
   #[view]
   public fun can_withdraw_terminated_funds(
       schedule_obj: Object<Schedule>
   ): bool acquires Schedule {
       let schedule_addr = object::object_address(&schedule_obj);
       if (!exists<Schedule>(schedule_addr)) {
           return false
       };

       let schedule = borrow_global<Schedule>(schedule_addr);
       schedule.terminated_amount != 0
           && schedule.terminated_withdrawn != schedule.terminated_amount
           && schedule.withdrawn + schedule.terminated_withdrawn
               <= schedule.funded_amount
   }
   ```

2. In `vesting`, `create_schedule_calendar` and `create_schedule_interval` verify that the timestamps are in increasing order, but fail to check if they are in the future. Allowing past timestamps will render the vesting logic ineffective as the funds may be withdrawn immediately unlocked upon creation.
3. `vesting::create_schedule_calendar` does not enforce that each timestamp has a single unique amount. Before storing a new timestamp, check if it already exists in `unlock_timestamps`.
4. `vesting::create_schedule_interval` should check if `amount_to_fund != 0`. If `amount_to_fund` is zero before attempting to transfer, it may result in unnecessary transfer attempts. Similarly, `vesting::fund` should also check that `amount_to_fund` is not zero.
5. In `create_schedule_interval`, the function should check if `cliff_amount` is zero when `cliff_timestamp` is zero, as the concept of a *cliff* is irrelevant when `cliff_timestamp` is zero (since no special locking period exists). Thus, the `cliff_amount` should only exist if there is an actual cliff timestamp.

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Ensure the above checks are included in the codebase.

© 2025 Otter Audits LLC. All Rights Reserved.

Error Handling

OS-MGN-SUG-03

Description

1. In `vesting`, `create_schedule_calendar` and `create_schedule_interval` should add a separate check to ensure that the benefactor has enough `amount_to_fund` before proceeding with the transfer to improve error handling. If the benefactor does not have enough funds, the transfer will fail at runtime, abruptly terminating the execution. Similarly, `fund` should check, if the funder has enough coin to funds, to handle the potential errors if there is not enough cash to transfer
2. In `vesting::handle_claim_fee` should verify if `fee_payer` has enough coin to pay the fee before the transfer, to handle the potential error gracefully.

   >_ *move/airlock_vesting/sources/vesting.move* RUST

   ```rust
   /// @notice Handle paying claim fees.
   /// @param fee_payer The payer of the fee.
   /// @param vesting The vesting object.
   fun handle_claim_fee<PaymentCoinType>(
       fee_payer: &signer, vesting: &Vesting
   ) {
       if (vesting.claim_fee != 0) {
           assert!(vesting.fee_collector != @0x0, ERR_INVALID_FEE_COLLECTOR);
           aptos_account::transfer_coins<PaymentCoinType>(
               fee_payer, vesting.fee_collector, vesting.claim_fee
           );
       }
   }
   ```

3. `vesting_merkle::handle_payment` should check if the `vesting_signer` has enough coins to withdraw before proceeding with the token transfer to gracefully handle potential errors related to insufficient balance.
4. In `vesting_merkle`, functions such as `terminate_schedule`, `withdraw_calendar`, `withdraw_interval`, `transfer_beneficiary`, `get_vested_amount_interval`, and `get_vested_amount_calendar` operate on a `root_index` within `merkle_roots` without first verifying its existence. If `root_index` does not exist in `merkle_roots`, the functions may attempt to access an invalid index, resulting in an abort. Ensure the functions first check whether `root_index` exists in `merkle_roots`.

Remediation

Modify the code to ensure proper error handling is implemented.

© 2025 Otter Audits LLC. All Rights Reserved.

Utilization of incorrect Comparison operators

OS-MGN-SUG-04

Description

Currently in `vesting`, both `cancel_schedule_calendar` (shown below) and `can_cancel_calendar` utilize `<=` to compare the `now_seconds` (current timestamp) with the last timestamp in the `unlock_timestamps` vector. This may result in a situation where the schedule is considered cancelable even after it has reached the fully unlocked state where there is nothing left to cancel.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
public entry fun cancel_schedule_calendar([...]) acquires Calendar, Schedule, Vesting {
    [...]
    let now_seconds = timestamp::now_seconds();
    assert!(
        now_seconds
            <= *vector::borrow(
                &calendar.unlock_timestamps,
                vector::length(&calendar.unlock_timestamps) - 1
            ),
        ERR_ALREADY_FULLY_UNLOCKED
    );[...]
}
```

`calc_vested_amount_calendar` treats any timestamp less than or equal to the final timestamp as "unlocked." If the schedule has already reached the final unlock timestamp, it indicates that there are no more amounts to unlock, so there is nothing left to cancel. Changing the condition to `<` ensures consistency between the logic for canceling and the calculation of vested amounts.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
public entry fun cancel_schedule_interval(
    benefactor: &signer, interval_obj: Object<Interval>
) acquires Interval, Schedule, Vesting {
    [...]
    let interval = borrow_global<Interval>(schedule_addr);
    let fully_unlocked_timestamp =
        interval.unlock_timestamp
            + (interval.number_of_periods * interval.period_length);
    let now_seconds = timestamp::now_seconds();
    assert!(now_seconds <= fully_unlocked_timestamp, ERR_ALREADY_FULLY_UNLOCKED);
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Additionally, `cancel_schedule_interval` (shown above) and `can_cancel_interval` also utilize `<=` to compare the `now_seconds` and `fully_unlocked_timestamp`, resulting in a similar situation where the schedule is considered cancelable even after it has reached the fully unlocked state where there is nothing left to cancel.

Remediation

utilize `<` instead of `<=` when comparing timestamps in the above functions to prevent canceling schedules that are already fully unlocked.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-MGN-SUG-05

Description

1. Functions such as `vesting_merkle::withdraw_interval` and `vesting::transfer_beneficiary_address`, which modify the state should emit events for proper logging of state modifications and for providing better integration with off-chain services and user interfaces.
2. Replace `ERR_NOT_FUNDED` with more specific error codes in `withdraw_terminated_funds` to improve the clarity and precision of the error messages when the withdrawal fails to meaningfully describe the point of failure.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
public entry fun withdraw_terminated_funds<CoinType>(
    benefactor: &signer, schedule_obj: Object<Schedule>
) acquires Schedule, Vesting {
    [...]
    let schedule = borrow_global_mut<Schedule>(schedule_addr);
    assert!(schedule.terminated_amount != 0, ERR_NOT_FUNDED);
    assert!(
        schedule.terminated_withdrawn != schedule.terminated_amount, ERR_NOT_FUNDED
    );
    [...]
}
```

3. `vesting::create_schedule_interval` may be optimized by utilizing the already existing `vesting_addr` value instead of recomputing it in `ScheduleCreatedEvent` emission.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
public fun create_schedule_interval<CoinType>([...]): Object<Interval> acquires Vesting {
    [...]
    event::emit(
        ScheduleCreatedEvent {
            schedule_addr,
            vesting_addr: object::object_address(&vesting_obj),
            [...]
        }
    );
    [...]
}
```

4. All `struct` fields of vector type should have a reasonable size limit to prevent potential denial-of-service attacks on vector operations.

© 2025 Otter Audits LLC. All Rights Reserved.

5. `all_vestings` in `vesting` and `vesting_merkle` retrieves all `Vesting` objects stored under `Airlock`. However, this approach has significant performance and reliability concerns due to the potentially large number of `vesting` objects stored and due to the fact that `smart_vector::to_vector` is a costly operation. This is especially concerning as anyone may add a `Vesting` to an `Airlock`, implying that the size of `vestings` may grow indefinitely.

   >_ *move/airlock_vesting/sources/vesting.move* RUST

   ```rust
   #[view]
   public fun all_vestings(): vector<Object<Vesting>> acquires Airlock {
       let airlock = borrow_global<Airlock>(package::resource_account_address());
       smart_vector::to_vector(&airlock.vestings)
   }
   ```

6. Moving `DistributionState` in `get_or_create_distribution_state` and `move_distribution_state_to_global_state` is unnecessary. Instead, a helper function may be implemented to ensure its existence, allowing it to be borrowed within the main function flow.

Remediation

Implement the above-mentioned suggestions.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Redundancy

OS-MGN-SUG-06

Description

1. `vesting::create_schedule_interval` already checks that the `total_amount` is exactly equal to `amount`. This check implicitly guarantees that `unlocked_at_start + cliff_amount` will not exceed `amount`. If `unlocked_at_start + cliff_amount` is more than `amount`, then `total_amount` would also be incorrect, and the assertion `total_amount == amount` will fail anyway, rendering this check unnecessary.

   >_ *move/airlock_vesting/sources/vesting.move* RUST

   ```rust
   public fun create_schedule_interval<CoinType>([...]): Object<Interval> acquires Vesting {
       [...]
       assert!(
           unlocked_at_start + cliff_amount <= amount,
           ERR_INVALID_AMOUNT
       );
       [...]
       let total_amount =
           get_total_amount_for_interval(
               unlocked_at_start,
               cliff_amount,
               number_of_periods,
               tokens_per_period
           );
       assert!(total_amount == amount, ERR_INVALID_AMOUNT);
       [...]
   }
   ```

2. In `vesting`, both `create_schedule_calendar` and `create_schedule_interval` borrow the `Vesting` object twice creating unnecessary duplication. It first does an immutable borrow via `borrow_global` and then a mutable borrow via `borrow_global_mut`. Instead, it will be appropriate to borrow `Vesting` mutably once at the beginning.
3. In `withdraw_terminated_funds`, `withdraw_calendar` and `withdraw_interval` within `vesting`, the `ERR_INVALID_WITHDRAWAL` assertion is duplicated because the exact same check is performed within `withdraw_to_beneficiary` and `withdraw_terminated_to_benefactor`.

© 2025 Otter Audits LLC. All Rights Reserved.

4. `vesting::withdraw_terminated_funds` retrieves the benefactor's address twice via `signer::address_of(benefactor)`. This is unnecessary, as the benefactor's address remains the same throughout execution and may be acquired only once.

>_ *move/airlock_vesting/sources/vesting.move* RUST

```rust
public entry fun withdraw_terminated_funds<CoinType>(
    benefactor: &signer, schedule_obj: Object<Schedule>
) acquires Schedule, Vesting {
    [...]
    assert_is_benefactor(vesting.benefactors, signer::address_of(benefactor));
    [...]
    withdraw_terminated_to_benefactor<CoinType>(
        &vesting_signer,
        schedule,
        signer::address_of(benefactor),
        amount_to_terminate
    );
}
```

Remediation

Remove the redundant code instances.

© 2025 Otter Audits LLC. All Rights Reserved.

Unutilized Code

OS-MGN-SUG-07

Description

1. In both `calc_vested_amount_calendar` and `calc_vested_amount_interval` within `vesting` and `vesting_merkle`, the use of `math64::min` to ensure the current `timestamp` is grater than than `terminated_timestamp` is unnecessary since `terminated_timestamp` will never be in the future. The protocol ensures that when a termination occurs, `terminated_timestamp` is recorded as one or the current timestamp.

   >_ *move/airlock_vesting/sources/vesting.move* RUST

   ```rust
   fun calc_vested_amount_interval([...]): u64 {
       let timestamp = timestamp::now_seconds();
       let final_timestamp =
           if (terminated_timestamp == 0) timestamp
           else math64::min(terminated_timestamp, timestamp);
       [...]
   }
   ```

2. The utilization of `math64::min` in `vesting::calc_vested_amount_interval` is unnecessary, based on the constraints of the different scenarios within the function.
   - In the case of `final_timestamp` < `unlock_timestamp`, the amount is 0.
   - In the case of `final_timestamp` < `cliff_timestamp`, the amount is `unlocked_at_start`, which will always be less than or equal to `unlocked_at_start` + `cliff_amount` + (`number_of_periods` * `tokens_per_period`).
   - In the case of `final_timestamp` ≥ `cliff_timestamp`, the amount is `unlocked_at_start` + `cliff_amount` + (`periods` * `tokens_per_period`), but `periods` is limited to `number_of_periods`. In the worst case, `periods` is equal to `number_of_periods`, so the amount will be equal to `get_total_amount_for_interval`.

   Thus, there is no possibility that `get_total_amount_for_interval` will be less than the amount.
3. The signer in `vesting::create_vesting` is not utilized, and in `vesting_merkle`, the `schedules` field in `Vesting` is un-utilized. Also, `pos` in `merkle_proof::get_proof` is unnecessary and may be removed.
4. There are multiple un-utilized constant variables throughout the code, such as `ERR_UNAUTHORIZED` in `vesting`, which may be removed.

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Remove any unutilized code.

© 2025 Otter Audits LLC. All Rights Reserved.

Unnecessary Assert Check

OS-MGN-SUG-08

Description

The `ERR_INVALID_WITHDRAWAL` asserts in `vesting` will currently never trigger. In `withdraw_calendar` and `withdraw_interval` the assert check is `withdrawn` + `terminated_withdrawn` + `withdrawable_amount` $\leq$ `funded` and the possible outcomes of this assert are:

1. While `terminated_withdrawn` is 0 and `withdrawable_amount` is `vested` - `withdrawn` (which makes an assumption of `vested` $<$ `funded` because of `math64::min` usage) the check shortens to `withdrawn` + 0 + `vested` - `withdrawn` $\leq$ `funded`, and further to `vested` $\leq$ `funded`, which is always true due to the previous assumption.
2. While `terminated_withdrawn` is 0 and `withdrawable_amount` is `funded` - `withdrawn` (which makes an assumption of `funded` $\leq$ `vested` because of `math64::min` usage) the check shortens to `withdrawn` + 0 + `funded` - `withdrawn` $\leq$ `funded`, and further to $0 \leq 0$, which is always true.
3. While `terminated_withdrawn` is `funded` - `vested` (cancellation case) and `withdrawable_amount` is `vested` - `withdrawn` (which makes an assumption of `vested` $<$ `funded` because of `math64::min` usage) the check shortens to `withdrawn` + `funded` - `vested` + `vested` - `withdrawn` $\leq$ `funded`, and further to $0 \leq 0$, which is always true.
4. While `terminated_withdrawn` is `funded` - `vested` (cancellation case) and `withdrawable_amount` is `funded` - `withdrawn` (which makes an assumption of `funded` $\leq$ `vested` because of `math64::min` usage) the check shortens to `withdrawn` + `funded` - `vested` + `funded` - `withdrawn` $\leq$ `funded`, and further to `funded` $\leq$ `vested`, which is always true due to the previous assumption.
5. In case of `revoke`, there is nothing to withdraw.

In `withdraw_terminated_funds`, the only possible values for `terminated_withdrawn` is 0 or `terminated_amount`. For the assertion `schedule.terminated_withdrawn != schedule.terminated_amount` to hold, `terminated_withdrawn` must be 0 in this case. Partial withdrawals are not possible for terminated funds. The assert check in this case is `withdrawn` + `terminated_withdrawn` + `amount_to_terminate` $\leq$ `funded_amount` and the potential assertion outcomes are therefore as follows:

© 2025 Otter Audits LLC. All Rights Reserved.

1. While `amount_to_terminate` is `funded` - `vested` (cancellation case), which shortens to `withdrawn` + 0 + `funded` - `vested` $\leq$ `funded`, and further to `withdrawn` $\leq$ `vested`, which is always true because the withdraws are limited to vested amounts (or funded if less than vested).
2. While `amount_to_terminate` is `funded` - `withdrawn` (revoke case), which shortens to `withdrawn` + 0 + `funded` - `withdrawn` $\leq$ `funded`, and further to $0 \leq 0$, which is always true.

Remediation

Refactor the `ERR_INVALID_WITHDRAWAL` assert check in these functions.

© 2025 Otter Audits LLC. All Rights Reserved.

Potential Overflow Scenarios

OS-MGN-SUG-09

Description

In the current codebase, overflows may occur in multiple areas of the code. However, they require an administrator to set extremely large values. It is assumed that the administrator is not malicious and is knowledgeable about the protocol, ensuring reasonable parameter settings that prevent such cases. Below are examples where an overflow may occur:

1. `create_schedule_calendar` or `create_schedule_interval` will abort if the sum of amounts is greater than max u64.
2. `cancel_schedule_interval` or `can_cancel_interval` may abort if `interval.unlock_timestamp` + (`interval.number_of_periods` * `interval.period_length`) is more than max u64.
3. `vesting_merkle::calc_vested_piece_amount` may abort if `piece.amount` * `fully_vested_periods` is more than max u64.
4. `vesting_merkle::calc_vested_amount_interval` will abort if the sum of vested amounts is more than max u64.

Remediation

Add overflow checks and upscaling in potential overflow areas.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.